CHARUSAT®

अमृतं तु विद्या

CHAROTAR UNIVERSITY OF SCIENCE AND TECHNOLOGY

FACULTY OF TECHNOLOGY AND ENGINEERING

DEVANG PATEL INSTITUTE OF ADVANCE TECHNOLOGY AND RESEARCH

DEPARTMENT OF INFORMATION TECHNOLOGY

A.Y 2026 – 27 Semester – 3

LAB MANUAL

CSUC201 Fundamentals of Data Structures and Algorithms

DEPSTAR

Practical-6.1

Github Link: https://github.com/rudraaaajoshi/FDSA_BATCH_A/tree/main/Practical-6

Difficulty :

Understanding the top position and stack operations was a little difficult.

Error :

The stack condition was not checked properly before push and pop operations.

Solution:

I added checks for overflow and underflow before performing operations.

Outcome:

The stack successfully performed push, pop, and display operations correctly.

Code(Github) Link:

https://github.com/rudraaaajoshi/FDSA_BATCH_A/blob/main/Practical-6/pr-6.1.cpp

Output:

```
Tray added: 10
Tray added: 20
Tray added: 30
Stack: 30 20 10
Tray removed: 30
Stack: 20 10
Tray removed: 20
Stack: 10

Process returned 0 (0x0)   execution time : 1.061 s
Press any key to continue.
```

Key Question:

Problem-1

A circular array is one way to reuse freed slots without shifting all elements. Without implementing it, explain what change would need to be made to your front and rear pointer logic to make the array behave circularly, and what new condition would indicate a full array

Answer:

The front and rear pointers should move using modulo (% SIZE), so after reaching the last position they go back to index 0.A circular array is full when (rear + 1) % SIZE == front.
This allows freed spaces at the beginning to be reused without shifting elements.

Problem-6.2

Difficulty :

Managing dynamic memory and linked-list nodes was a little difficult.

Error :

The new page was not linked correctly with the previous page.

Solution:

I used next pointers to add and remove pages dynamically.

Outcome:

The browser history grows and shrinks successfully without a fixed size.

Code(Github) Link: https://github.com/rudraaaajoshi/FDSA_BATCH_A/blob/main/Practical-6/pr-6.2.cpp

Output:

```
Current page: Google
Current page: YouTube
Current page: Instagram
Current page: YouTube
Current page: Google
No page left
No history left

Process returned 0 (0x0)   execution time : 1.059 s
Press any key to continue.
```

Key Question:

Problem-2:

Your linked list stack has no fixed capacity limit. But in practice, can it still run out of space? What determines the actual limit, and why is that limit invisible in your code?

Answer:

A linked-list stack can still run out of space if the computer's available memory (RAM/heap) is exhausted.The actual limit depends on the available system memory and memory used by other programs.
It is invisible in the code because there is no fixed array size; memory is allocated dynamically when new nodes are created.

Practical-6.3

Difficulty :

Understanding operator priority and the order of postfix conversion was a little difficult.

Error :

Operators were sometimes added to the postfix expression in the wrong order.

Solution:

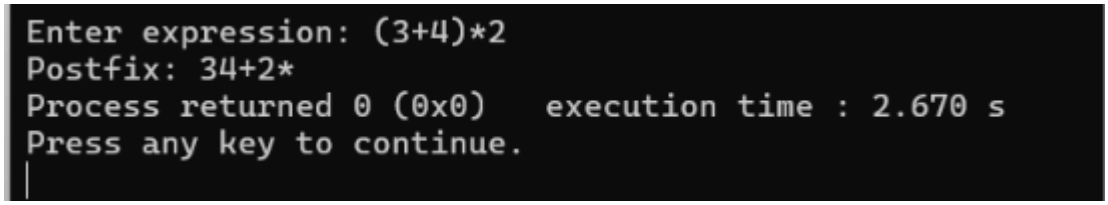
I used a stack to temporarily store operators and popped them according to their priority and brackets.

Outcome:

The infix expression was successfully converted into the correct postfix form.

Code(Github) Link: https://github.com/rudraaaajoshi/FDSA_BATCH_A/blob/main/Practical-6/pr-6.3.cpp

Output:



Key Question:

Problem-3

Operator precedence determines the order in which operators are placed into the output. Without tracing any expression, explain what would go wrong in your output if your solution treated all operators as having equal precedence.

Answer:

If all operators had equal precedence, the expression would be processed only from left to right.
This could put + before * or /, producing an incorrect postfix expression.
Therefore, operator precedence is needed to maintain the correct order of operations.